

Объектно ориентированное программирование на C#

Тема 24. Теория. Работа с JSON

Что такое JSON формат.

JSON (JavaScript Object Notation) - это легкий формат обмена данными, который используется для хранения и передачи структурированных данных.



JSON представляет собой текстовый формат, который состоит из пар "ключ-значение" и используется в различных языках программирования. Вот основная структура JSON:

```
package.json x
1  {
2    "name": "myapplication",
3    "description": "some description here",
4    "version": "0.0.1",
5    "private": true,
6    "scripts": {
7      "start": "node ./bin/www"
8    },
9    "dependencies": {
10     "express": "~4.12.2",
11     "jade": "~1.9.2"
12   }
13 }
```

Объекты (Objects).

Объект в JSON представляет собой неупорядоченное множество пар "ключ-значение".

Ключи являются строками, а значения могут быть строками, числами, логическими значениями, массивами, другими объектами или null.

Объекты обозначаются фигурными скобками {}.

Пример:

```
{  
  "имя": "John",  
  "возраст": 30,  
  "женат": false  
}
```

Массивы (Arrays).

Массив в JSON представляет собой упорядоченный список значений.

Значения в массиве могут быть строками, числами, логическими значениями, объектами, массивами или null.

Массивы обозначаются квадратными скобками [].

Пример:

```
[  
  "яблоко",  
  "банан",  
  "апельсин"  
]
```

Значения (Values).

Значения в JSON могут быть строками, числами, логическими значениями, объектами, массивами или null.

Примеры:

```
{  
  "Строка": "Привет, мир!",  
  "Число": 42,  
  "Логическое значение1": true,  
  "Логическое значение2": false,  
  "Null": null  
}
```

Ключи (Keys).

Ключи в JSON должны быть строками и заключаются в двойные кавычки.

Ключи используются для идентификации полей в объектах.

Пример:

```
{  
  "ключ1": "значение1",  
  "ключ2": "значение2"  
}
```

Запятые (Commas).

Запятые используются для разделения элементов в массивах и пар "ключ-значение" в объектах. Важно помнить, что после последнего элемента в объекте или массиве запятая не ставится.

Для чего используется JSON.

JSON является популярным форматом для обмена данными между различными приложениями и сервисами.

Одна из основных причин использования - универсальность, как вы знаете, языков программирования, разнообразных приложений и сервисов, написанных на этих языках еще больше и эти приложения должны как то обмениваться информацией о своем состоянии, для этого есть много способов, но один из популярных - использование общего, стандартизированного языка JSON.

Так же он используется еще по нескольким причинам:

Легкость чтения и записи: JSON имеет простой и понятный синтаксис, который легко читать и писать как человеку, так и компьютеру. Это делает его удобным для разработчиков.

Поддержка различных языков программирования: JSON может быть использован практически во всех языках программирования, так как множество современных языков предоставляют функции для работы с JSON, включая сериализацию (преобразование данных в JSON) и десериализацию (преобразование JSON обратно в структуры данных языка).

Легкость передачи данных по сети: JSON является текстовым форматом, что делает его идеальным для передачи данных по сети. Он легко сериализуется в строку и десериализуется на другом конце соединения. Это особенно важно для веб-сервисов и API.

Используется во множестве сценариев: JSON используется для передачи данных во многих сценариях, включая веб-разработку (как формат данных

для обмена данными между веб-сервером и клиентом), хранение конфигураций, обмен данными между мобильными приложениями и серверами, обмен данными между микросервисами и многие другие.

Поддержка разнообразных типов данных: JSON поддерживает разнообразные типы данных, включая строки, числа, булевы значения, массивы, объекты и null. Это позволяет представлять различные структуры данных в удобной и понятной форме.

Расширяемость: JSON позволяет встраивать пользовательские типы данных и обеспечивает гибкость в проектировании структур данных, что делает его мощным средством для представления различных сущностей и информации.

Из-за своей простоты и универсальности JSON широко применяется в различных областях разработки программного обеспечения и считается одним из стандартных форматов для обмена данными в веб-технологиях и не только.

Что касается **работы с JSON используя С#**.

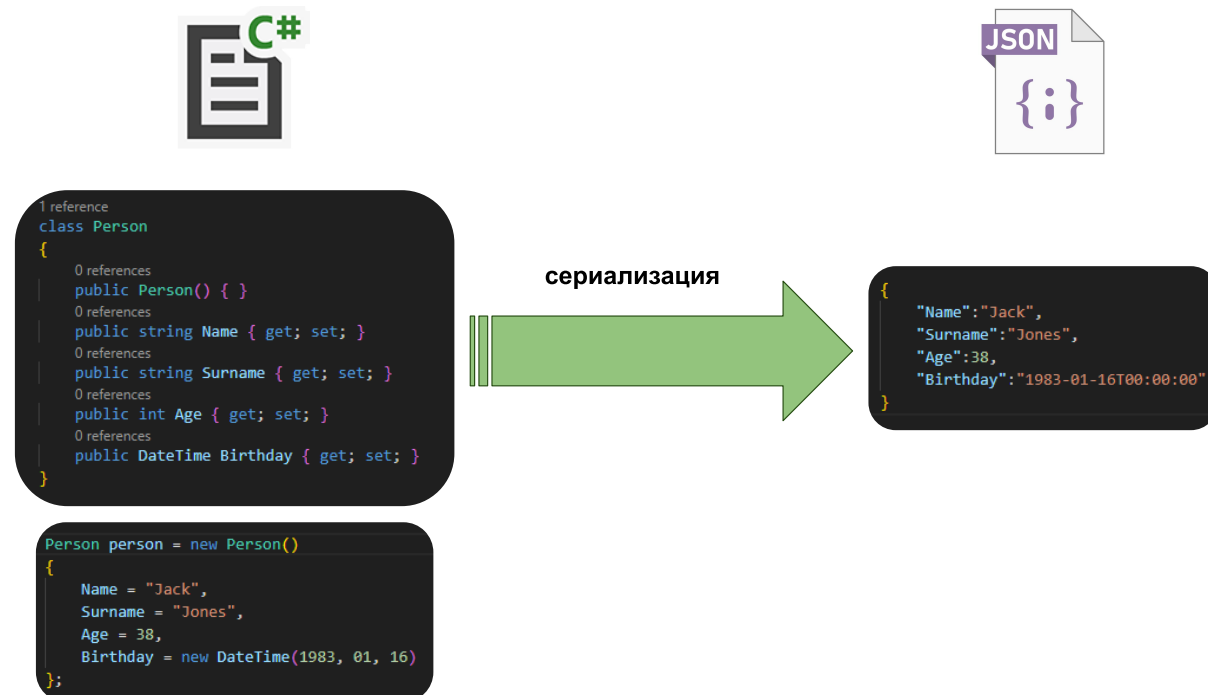
Сериализация и десериализация - это процессы преобразования данных между разными форматами или структурами, чтобы они могли быть сохранены, переданы или восстановлены в другой момент времени. Эти процессы широко используются в программировании, особенно при работе с данными, которые должны быть сохранены или переданы через сеть.

Рассмотрим каждый из них по отдельности.

Сериализация - это процесс преобразования структуры данных или объекта в формат, который можно легко сохранить или передать. Обычно это означает преобразование данных в текстовый или двоичный формат, который может быть записан в файл или передан через сеть.

В контексте JSON, например, сериализация означает преобразование объекта или структуры данных в JSON-строку.

Примеры других форматов для сериализации включают XML, бинарные форматы (например, Protocol Buffers), и другие.



Десериализация - это процесс восстановления данных из сериализованного формата обратно в исходную структуру данных или объект. То есть, это обратная операция сериализации.

В контексте JSON, десериализация означает преобразование JSON-строки обратно в объект или структуру данных, которую можно использовать в программе.

Для успешной десериализации данные в сериализованном формате должны соответствовать структуре данных, в которую они преобразовываются.

Прежде чем начать детально разбирать процессы сериализации и десериализации, рассмотрим простой пример сериализации объекта в JSON файл.

Для начала, нам необходим объект человека со следующими свойствами: Name, Surname, Age, Birthday отвечающие за имя, фамилию, возраст, дату рождения соответственно

Итак, класс:

```
class Person
{
    public string Name { get; set; }
    public string Surname { get; set; }
    public int Age { get; set; }
    public DateTime Birthday { get; set; }
}
```

Теперь в методе Main класса Program создадим объект класса Person и инициализируем его свойства значениями.

Также, допишите далее код, который ниже. На данный момент вникать в него не нужно, просто запустите и посмотрите как он отработает:

```
class Program
{
    public static void Main(string[] args)
    {
        Person person = new Person()
        {
            Name = "Jack",
            Surname = "Jones",
            Age = 38,
            Birthday = new DateTime(1983, 01, 16)
        };
        string personJson = JsonSerializer.Serialize(person,
typeof(Person));
        StreamWriter file = File.CreateText("person.json");
        file.WriteLine(personJson);
        file.Close();
    }
}
```

После запуска, ваш объект Person импортируется (сериализуется) JSON расположенный в папке проекта (...SandBox\bin\Debug\net6.0)

Поэкспериментируйте с разными объектами и их свойствами, а мы продолжаем.

Библиотека System.Text.Json.

System.Text.Json - это библиотека, встроенная в .NET Core и .NET 5 (и более поздние версии) для работы с JSON (JavaScript Object Notation).

System.Text.Json предоставляет средства для сериализации (преобразования объектов .NET в JSON) и десериализации (преобразования JSON в объекты .NET) данных.

Она является альтернативой библиотеке Newtonsoft.Json (также известной как JSON.NET), которая была широко используется в .NET Framework.

Преимущества System.Text.Json включают:

Производительность: Она была разработана с учетом производительности и может работать быстрее, чем Newtonsoft.Json, особенно в сценариях большого объема данных.

Интеграция: Она интегрирована непосредственно в .NET Core и .NET 5+, что делает ее стандартной библиотекой для работы с JSON.

Легкость использования: Она предоставляет удобный API для сериализации и десериализации объектов.

Сериализация объекта в JSON.

Сериализация - это процесс преобразования объекта C# в формат JSON.

Для этого вы можете использовать класс JsonSerializer:

```
using System.Text.Json;

public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}

class Program
{
    static void Main()
    {
        Person person = new Person
        {
            Name = "John",
            Age = 30
        };
        string json = JsonSerializer.Serialize(person);
        Console.WriteLine(json);
    }
}
```


Разнообразим наше приложение и рассмотрим сериализацию свойства с датой и словарем:

```
class Program
{
    static async Task Main()
    {
        Person person = new Person()
        {
            Name = "Jack",
            Surname = "Jones",
            Age = 38,
            Birthday = new DateTime(1983, 01, 16),
            Childrens = new Dictionary<int, string>{ [1] = "Sam", [2] =
"Alice", [3] = "Nick" }
        };
        string json = JsonSerializer.Serialize(person);
        Console.WriteLine(json);
    }
}

class Person
{
    public string Name { get; set; }
    public string Surname { get; set; }
    public int Age { get; set; }
    public DateTime Birthday { get; set; }

    public Dictionary<int, string> Childrens { get; set; }
}
```

В этом коде происходит следующее:

Создается экземпляр класса Person с именем person, и ему присваиваются различные свойства, такие как имя, фамилия, возраст, дата рождения и словарь детей.

Используется JsonSerializer.Serialize(person) для сериализации объекта person в строку JSON. Этот метод преобразует объект person в его JSON-представление.

Полученная строка JSON записывается в консоль с помощью Console.WriteLine(json).

Итак, весь этот код выполняет сериализацию объекта person в формат JSON и выводит JSON-строку в консоль. В результате выполнения кода на консоль будет выведено JSON-представление объекта person.

Запись JSON в файл:

На данный момент, пока мы не владеем работой с файловыми потоками, запишем нашу строку json в файл при помощи класса File.

```
File.WriteAllText("person.json", json);
```

Десериализация JSON в объект.

Десериализация - это обратный процесс, при котором JSON-данные преобразуются обратно в объект C#. В System.Text.Json для этой цели также используется класс JsonSerializer:

```
class Person
{
    public string Name { get; set; }
    public string Surname { get; set; }
    public int Age { get; set; }
    public DateTime Birthday { get; set; }

    public Dictionary<int, string> Childrens { get; set; }
}

class Program
{
    public static void Main()
    {
        string json = File.ReadAllText("person.json");
        Person person = JsonSerializer.Deserialize<Person>(json);
        Console.WriteLine(
            $"Name: {person.Name}\n" +
            $"Surname: {person.Surname}\n" +
            $"Age: {person.Age}\n" +
            $"BirthDay: {person.Birthday}\n" +
            $"Childrens: {person.Childrens[1]}\n" +
            $"Childrens: {person.Childrens[2]}\n" +
            $"Childrens: {person.Childrens[3]}\n");
    }
}
```

В этом примере мы берем строку JSON и десериализуем ее в объект типа Person.

Настройка сериализации и десериализации.

System.Text.Json предоставляет множество опций для настройки процесса сериализации и десериализации, таких как форматирование, игнорирование определенных полей, обработка дат и т. д. Вы можете настраивать JsonSerializerOptions перед выполнением сериализации или десериализации.

```
public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}

class Program
{
    public static void Main()
    {
        Person person = new Person { Name = "John", Age = 30 };
        JsonSerializerOptions options = new JsonSerializerOptions
        {
            WriteIndented = true // Добавляет отступы для красивого
форматирования
        };
        string json = JsonSerializer.Serialize(person, options);
        Console.WriteLine(json);
    }
}
```

Это только базовые примеры работы с System.Text.Json. В зависимости от ваших потребностей, вам может потребоваться более сложная настройка. Помимо этого, System.Text.Json обычно более производительный по сравнению с другими библиотеками для работы с JSON в C#.

Вот некоторые из наиболее используемых свойств:

WriteIndented (тип: bool).

Используется для установки, “следует ли добавлять отступы и разделители строк?”, чтобы JSON был более читаемым.

Пример: options.WriteIndented = true;

Принимает булевы типы.

PropertyNameingPolicy (тип: JsonNamingPolicy).

Определяет, как имена свойств C# класса преобразуются в имена свойств JSON.

Пример: `options.PropertyNamingPolicy = JsonNamingPolicy.CamelCase;`

Переведет ключи в CamelCase

DefaultIgnoreCondition (тип: JsonIgnoreCondition).

Указывает, какие свойства C# класса должны быть автоматически игнорированы при сериализации, например, если они имеют значение `null`.

Пример:

`options.DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull;`

Проигнорирует все свойства со значением `null`

NumberHandling (тип: JsonNumberHandling).

Позволяет настраивать обработку чисел, включая использование двойной точности, десятичной точности и других форматов.

Пример:

`options.NumberHandling = JsonNumberHandling.AllowReadingFromString;`

DictionaryKeyPolicy (тип: JsonNamingPolicy).

Определяет, как имена ключей словарей преобразуются в имена ключей JSON.

Пример: `options.DictionaryKeyPolicy = JsonNamingPolicy.CamelCase;`

Encoder (тип: JavaScriptEncoder).

Управляет кодированием символов в строковых значениях.

Пример: `options.Encoder = JavaScriptEncoder.UnsafeRelaxedJsonEscaping;`

Пример использования JsonSerializerOptions:

```
JsonSerializerOptions options = new JsonSerializerOptions
{
    WriteIndented = true,
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
    DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,
    NumberHandling = JsonNumberHandling.AllowReadingFromString,
```

```
DictionaryKeyPolicy = JsonNamingPolicy.CamelCase,  
Encoder = JavaScriptEncoder.UnsafeRelaxedJsonEscaping  
};
```

Передаются параметры при помощи перегрузки метода `Serialize`, где первым параметром мы передаем сериализуемый экземпляр класса, а вторым - сами параметры настроек.

```
string json = JsonSerializer.Serialize(person, options);
```

Вот какой будет вывод:

```
Person person = new Person()  
{  
    Name = "Bogdan",  
    Surname = "Chyorny",  
    Age = 24,  
    Birthday = new DateTime(1999, 1, 6),  
    Childrens = null  
};  
  
JsonSerializerOptions options = new JsonSerializerOptions  
{  
    WriteIndented = true,  
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase,  
    DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,  
    NumberHandling = JsonNumberHandling.AllowReadingFromString,  
    DictionaryKeyPolicy = JsonNamingPolicy.CamelCase,  
    Encoder = JavaScriptEncoder.UnsafeRelaxedJsonEscaping  
};  
  
string json = JsonSerializer.Serialize(person, options);  
Console.WriteLine(json);
```

Консоль отладки Microsoft Visual Studio

```
{  
  "name": "Bogdan",  
  "surname": "Chyorny",  
  "age": 24,  
  "birthday": "1999-01-06T00:00:00"  
}
```

C:\Users\Богдан\source\repos\SandBox\
Нажмите любую клавишу, чтобы закрыть

Как вы видите, формат сериализации изменился, он выстроился в понятную структуру, а не строкой, ключи преобразовались из `PascalCase` в `CamelCase`, значения `null` не отображаются, разрешено чтение чисел из строк и другие настройки.